

LED PWM Kontrolü ve Buton ile State Machine

TIM2 tabanlı yazılımsal PWM, EXTI/NVIC kesme sistemi ve enum + switch-case ile yazılmış 4 durumlu bir state machine'in uçtan uca çalışma mantığı.

01 Mikrodenetleyici genel bakış

02 Saat sistemi (clock tree)

03 CubeMX yapılandırması

04 TIM2 ile yazılımsal PWM

05 EXTI + NVIC ile buton kesmesi

06 State machine tasarımı

07 Tam kod ve hızlı tekrar

08 Test: 12 soru, cevaplı

Blok diyagram ve pinout, STM32F401xB/xC veri sayfasından (DS9716 Rev 11) alınmıştır; diğer şemalar bu proje için çizilmiştir.

01

Mikrodenetleyici genel bakış

Hangi çip, hangi bacaklar, hangi çevre birimleri. Kodu okumadan önce elimizdeki donanımı tanıyoruz.

STM32F401CCU6 künyesi

Çekirdek	Cortex-M4 + FPU, maks. 84 MHz
Flash	256 KB
SRAM	64 KB
Paket	UFQFPN48 (7×7 mm)
Zamanlayıcı	TIM1 + 7 genel amaçlı
Bu projede saat	HSI = 16 MHz (PLL kapalı)

SOMUT KARŞILIK

84 MHz'lik bir çipi 16 MHz'de çalıştırmak, 240 km/h yapan arabayı şehir içinde 60'ta sürmek gibi. Bir LED'i soldurmak ve bir butonu okumak için motoru sonuna kadar açmaya gerek yok.

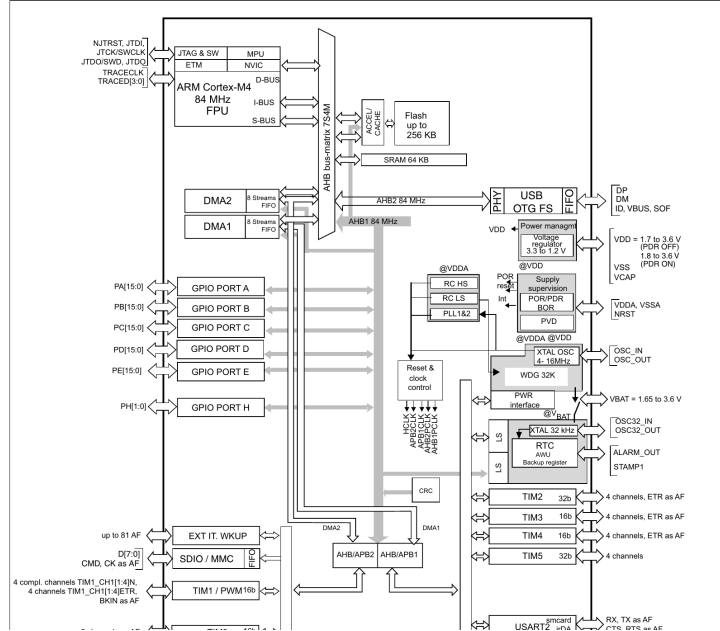
Kazanç: daha az güç, çok daha basit
SystemClock_Config, Flash için bekleme durumu yok.

İç blok diyagramı

Description

STM32F401xB STM32F401xC

Figure 3. STM32F401xB/STM32F401xC block diagram



DIYAGRAMDA NEREYE BAKMALI

- 1 Sol kolon: **GPIO PORT A** ve **GPIO PORT C** – PA0 ve PC13 buradan çıkıyor.
- 2 Sağ alt: **TIM2 32b** satırı – APB1 hattına bağlı, projedeki tek zamanlayıcı.
- 3 Sol üst: **NVIC** kutusu – hem TIM2 hem EXTI kesmesi CPU'ya buradan giriyor.
- 4 Sol alt: **EXT IT. WKUP** – EXTI hatlarının çipteki karşılığı.

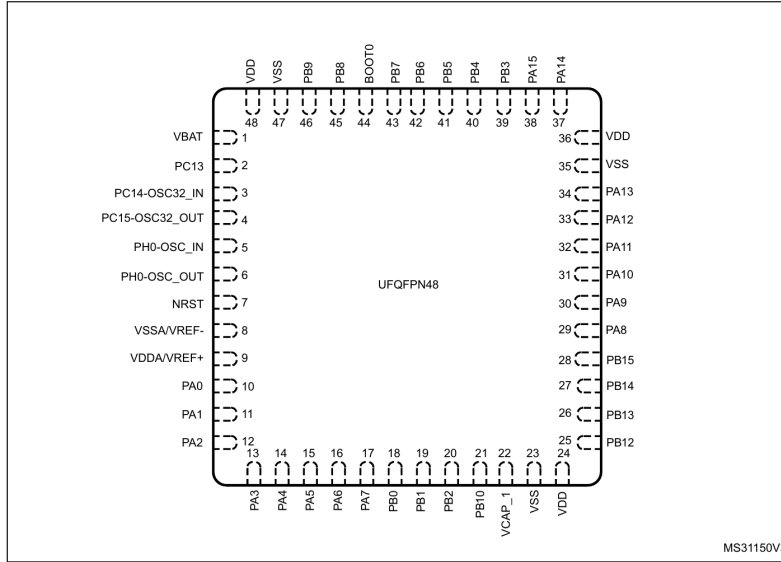
Kaynak: STM32F401xB/xC Datasheet, DS9716 Rev 11, Figure 3

UFQFPN48 pin yerleşimi

STM32F401xB STM32F401xC

Pinouts and pin description

Figure 11. STM32F401xB/STM32F401xC UFQFPN48 pinout



1. The above figure shows the package top view.

Pin 2 – PC13

LED çıkışı. Sol kenarda, Üstten ikinci bacak.

Pin 10 – PA0

KEY girişi. Yine sol kenarda, VDDA/VREF+ bacağının hemen altında.

Şema Üstten görünüşdür. Çipi kartla karşılaştırırken pin 1 nokta işaretini referans alın.

Kaynak: DS9716 Rev 11, Figure 11

Aktif-low LED bağlantısı

Black Pill kartında LED'in anodu 3.3 V'a, katodu PC13'e bağlı. Akım pinden dışarı değil, pinin **içine** akar. Yani devre ancak pin LOW iken tamamlanır.

PIN LOW = LED YANIK

```
HAL_GPIO_WritePin(LED_GPIO_Port,  
                  LED_Pin, GPIO_PIN_RESET);
```

Katot 0 V'a çekilir, 3.3 V üzerinden akım geçer.

PIN HIGH = LED SÖNÜK

```
HAL_GPIO_WritePin(LED_GPIO_Port,  
                  LED_Pin, GPIO_PIN_SET);
```

Pinin iki ucu da 3.3 V; potansiyel farkı yok, akım yok.

Bu ters mantık ilerideki PWM kodunda kritik: **pwm_phase < brightness** koşulu sağlandığında pini SET değil RESET yapıyoruz.

02

Saat sistemi (clock tree)

TIM2'nin kaç Hz'de saydığını bilmeden PWM frekansı hesaplanamaz. Her şey saat ağacının tepesinden başlar.

HSI'den TIM2CLK'a



Sınavlık kural: APB1 prescaler 1 ise $TIMxCLK = PCLK1$. Prescaler 1'den büyükse $TIMxCLK = 2 \times PCLK1$. Burada bölen 1 olduğu için çarpan devreye girmiyor.

Saat ağacı = fabrikanın şaft hattı

BENZETME

Eski bir fabrikada tavanda tek bir ana şaft döner. Her tezgâh bu şafta kayış ve kasnakla bağlanır; kasnak oranı o tezgâhın hızını belirler.

HSI = motor. SYSCCLK = ana şaft. AHB ve APB prescaler'ları = kasnaklar. TIM2 = tezgâhlardan biri. Motor hızını değiştirirseniz bütün tezgâhlar etkilenir; sadece kasnağı değiştirirseniz yalnız o tezgâh.

PRATİKTE NE DEMEK

- PLL'i açıp 84 MHz'e çıkıydınız, **aynı PSC/ARR değerleriyle** LED 5,25 kat hızlı solardı.
- Bu yüzden "PWM frekansım neden yanlış?" sorusunun cevabı çoğu zaman kodda değil, CubeMX'in Clock Configuration sekmesindedir.
- 16 MHz'de Flash bekleme durumu gerekmez: **FLASH_LATENCY_0**. Yüksek frekansta bu değer artar; unutulursa çip kilitlenir.

Neden PLL kullanmadık? PLL saati 84 MHz'e çıkarmak içindir. Bir LED soluma efekti ve tek buton okuması için gereksiz; 16 MHz HSI hem yeterli hem de kurulumu çok daha basit.

03

CubeMX yapılandırması

Grafik arayüzde yapılan her ayar .ioc dosyasına yazılır ve "Generate Code" ile HAL init fonksiyonlarına dönüşür.

GPIO ayarları

PIN	KULLANIM	MOD	PULL
PC13	LED çıkışı	GPIO_Output (Push-Pull)	No pull
PA0	KEY girişi	GPIO_EXTI0 (falling edge)	Pull-up

NEDEN PULL-UP

Butona basılmadığında PA0 hiçbir yere bağlı değildir; havada kalan giriş pini antene dönüşür ve rastgele HIGH/LOW okur (floating). Dahili pull-up direnci pini boştayken 3.3 V'a "yapıştırır". Basınca pin GND'ye çekilir, temiz bir HIGH→LOW geçişi oluşur.

NEDEN FALLING EDGE

Basma anı düşen kenar, bırakma anı yükselen kenardır. Sadece falling seçilince her basış tek olay üretir; "both edges" seçilseydi bir basıp bırakma iki kesme doğururdu.

TIM2 ve NVIC ayarları

TIM2

Saat kaynağı	Internal Clock – 16 MHz
Prescaler (PSC)	15 → bölen 16
Period (ARR)	99 → 100 adım
Counter Mode	Up
Başlatma	HAL_TIM_Base_Start_IT()

NVIC

EXTI0_IRQn	Aktif • öncelik 0
TIM2_IRQn	Aktif • öncelik 0

İkisi de aynı öncelikte olduğu için biri diğerini kesemez, sıraya girerler. LED soluma için bu yeterli.

USER CODE Bu projede main.c'de CubeMX'in **USER CODE BEGIN/END** etiketleri yoktu. Bir pin ayarı değiştirip "Generate Code" deneşeydi PWM, fade ve state machine kodunun tamamı silinebilirdi. Kural: elle düzenlenen HAL projelerinde bu etiketler asla silinmez, tüm özel kod aralarına yazılır.

04

TIM2 ile yazılımsal PWM

Zamanlayıcı sadece düzenli aralıklarla kesme üretir; pini yakıp söndürme işini kod yapar.

Donanımsal PWM yerine yazılımsal PWM

DONANIMSAL (kullanılmadı)

TIM2'nin CH1..CH4 çıkışları belirli pinlere bağlıdır. CCR registerine bir sayı yazarsınız, darbe genişliğini donanım üretir. CPU hiç uğraşmaz.

Sorun: PC13, TIM2'nin PWM çıkış kanallarından biri değil. Karttaki LED donanım PWM'e bağlanamıyor.

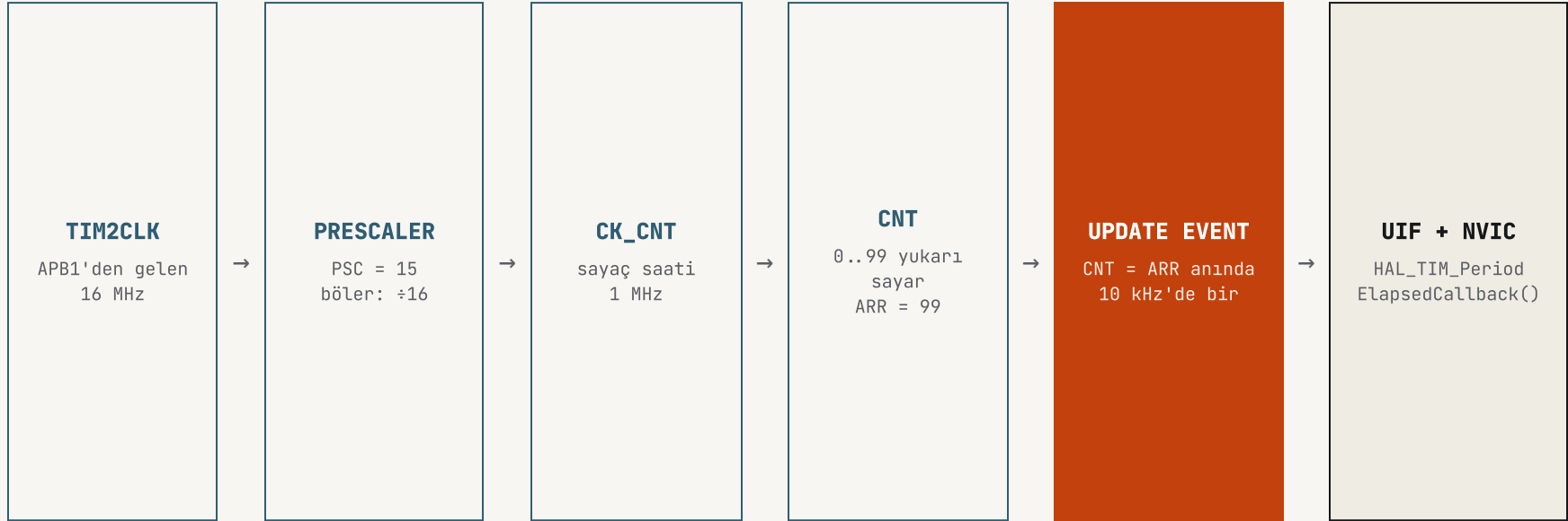
YAZILIMSAL (bu proje)

TIM2 yalnızca metronom: her 100 μ s'de bir kesme atar. Kesme içinde bir sayaç (pwm_phase) döner ve pin elle SET/RESET yapılır.

Bedeli: saniyede 10.000 kesme, yani sürekli CPU yükü. Kazancı: **herhangi bir** GPIO pininde PWM.

Somut: Donanımsal PWM, kurduktan sonra kendi kendine çalan bir müzik kutusudur. Yazılımsal PWM ise metronom eşliğinde her vuruşta elle tuşa basmaktır – aynı sesi çıkarır, ama eliniz sürekli meşguldür.

TIM2 iç zinciri: PSC → CNT → ARR → UEV



Zincirin ilk beş kutusu **donanımda** kendiliğinden döner. Biz sadece `HAL_TIM_Base_Start_IT(&htim2)` diyerek çarkı başlatıyoruz.

UIF = Update Interrupt Flag. Donanım bayrağı kaldırır, NVIC bunu CPU'ya iletir, HAL bayrağı temizleyip callback'imizi çağırır.

Prescaler ve ARR'ı somutlaştırmak

PRESCALER = dişli kutusu

Motor saniyede 16 milyon tur atıyor ve sayacın bunu takip etmesini istemiyorsunuz. Araya 16:1 dişli koyarsınız: motor 16 tur atınca çıkış mili 1 tur atar. $PSC = 15$ tam olarak budur – **bölen = $PSC + 1$.**

Neden +1? Çünkü $PSC = 0$ "bölme yok" demektir; register sıfırdan başlar.

ARR = kilometre sayacının turu

Eski araba kilometre sayacı 999999'a gelince 000000'a döner. ARR o "en büyük sayı"dır. CNT 99'a gelip sıfıra döndüğü an bir tur bitmiştir ve zil çalar: **Update Event.**

Adım sayısı = $ARR + 1 = 100$. Sıfır da bir adımdır.

TEK CÜMLE

Prescaler ne kadar sık sayılacağını, ARR kaç kadar sayılacağını belirler. Kesme frekansı ikisinin çarpımına bölünür.

Frekans hesabı, dört adım

01 • sayaç saati

$16.000.000 \div (15+1)$

1 MHz

02 • kesme frekansı

$1.000.000 \div (99+1)$

10 kHz

03 • PWM periyodu

$100 \text{ adım} \times 100 \mu\text{s}$

10 ms

04 • PWM frekansı

$1 \div 10 \text{ ms}$

100 Hz

CK_CNT – CNT bu hızda artar

her 100 μs 'de bir callback

pwm_phase bir tur atar

göz titremeyi fark etmez

Eşik bilgisi: insan gözü yaklaşık 60-80 Hz üzerinde kırpışmayı ayırt edemez. 100 Hz güvenli tarafta; 30 Hz'e düşseydi LED gözle görülür şekilde titrerdi.

Zaman ekseninde: CK_CNT, CNT, Update Event

TIM2CLK



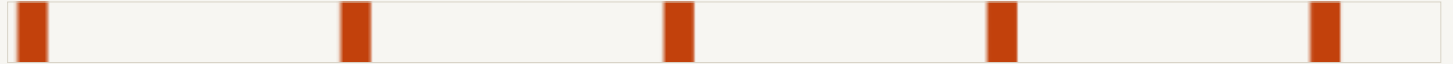
CK_CNT
(örnekte ÷4)



CNT
(ARR = 1)



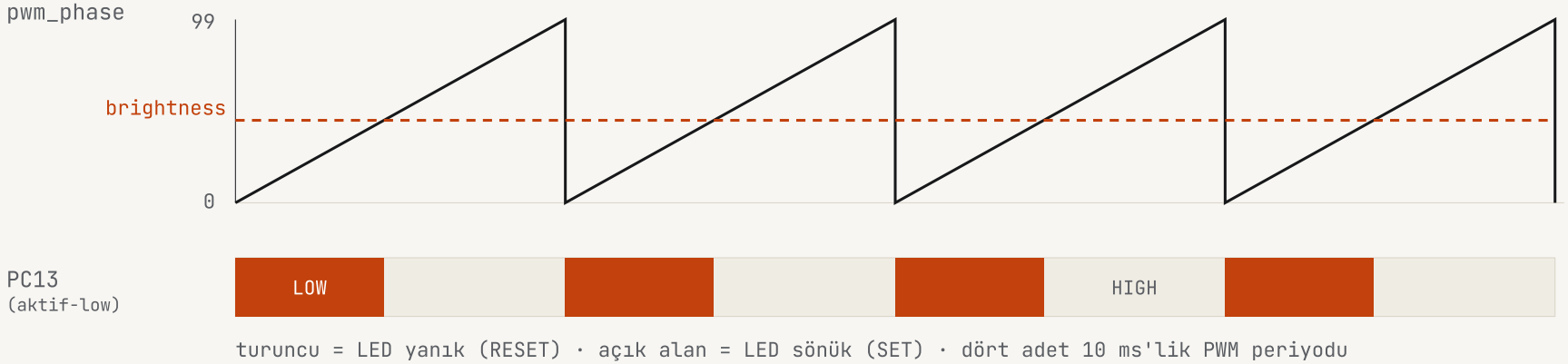
UEV



CNT, ARR'a ulaştı → 0'a döner, UIF bayrağı set edilir, kesme tetiklenir

Okunabilirlik için burada bölme oranı 1:4 ve ARR = 1 çizildi. Gerçek projede oran 1:16 (PSC = 15) ve ARR = 99'dur; mantık birebir aynı, sadece süreler uzun.

pwm_phase ve brightness ilişkisi



Okuma biçimi: testere dişi her turda 0'dan 99'a tırmanır. Kesikli çizginin **altında** kaldığı sürece LED yanar. Çizgiyi yukarı kaydırmak (`brightness`'ı artırmak) turuncu blokları genişletir; göz bunu "daha parlak" olarak okur.

PWM üreten kısım

```
/* TIM2 her 100 us'de bir cagirir */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    /* ...fade mantigi... */

    pwm_phase++;
    if (pwm_phase ≥ 100) { pwm_phase = 0; }

    if (pwm_phase < brightness)
        HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin,
                           GPIO_PIN_RESET); // yanar
    else
        HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin,
                           GPIO_PIN_SET);   // soner
}
```

- 1 Fonksiyon saniyede 10.000 kez çalışır. İçine uzun iş koymak yasak: döngü, delay, printf yok.
- 2 **pwm_phase** 0..99 arası sürekli döner; PWM'in yatay eksenidir.
- 3 **brightness** 0..100 arası eşiktir; grafikteki yatay kesikli çizgidir.
- 4 **brightness = 0** → koşul hiç sağlanmaz, LED hep sönmük. **brightness = 100** → hep sağlanır, LED tam parlak.

HAL bu callback'i tüm timer'lar için ortak çağırır. Projede tek timer olduğundan htim kontrolü yapılmadı; ikinci bir timer eklenirse `if (htim->Instance == TIM2)` şart olur.

05

EXTI + NVIC ile buton kesmesi

PA0'a basılmasından bizim switch-case'imizin çalışmasına kadar sinyalin izlediği yol.

Kesme zinciri: bacadan koda



Üst sıra çipte kendiliğinden olur. Alt sıradaki ilk iki fonksiyonu CubeMX ve HAL yazar; elle dokunduğumuz tek yer en sağdaki callback'tir.

Kesme = kapı zili, polling = pencereden bakmak

POLLING

Misafir gelecek diye her on saniyede bir pencereden bakarsınız. Çoğu bakış boşa gider; Üstelik tam bakmadığınız anda gelirse kaçırsınız.

```
while (1) {  
    if (HAL_GPIO_ReadPin(...)) { ... }  
}
```

KESME

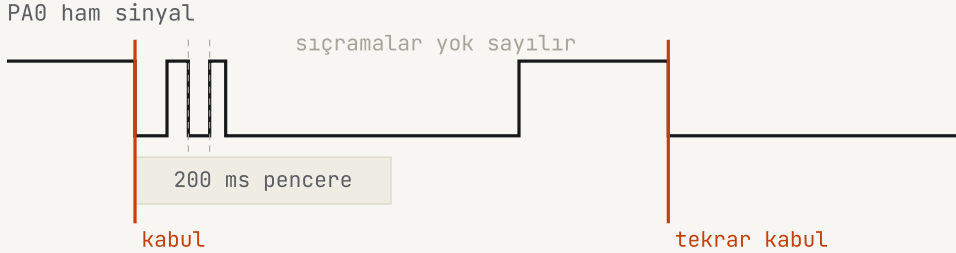
Zili takarsınız ve işinize dönersiniz. Zil çaldığı an ne yapıyorsanız bırakıp kapıya gider, sonra kaldığınız yerden devam edersiniz.

```
while (1) { }    // bos  
// her sey callback'lerde
```

BENZETMENİN SINIRI

Kapıya giderken elinizdeki tepsiyi masaya bırakmanız gerekir; CPU da register'larını yığına (stack) kaydeder. Bu yüzden kesme fonksiyonları kısa olmalı: kapıda uzun sohbet ederseniz mutfakta bir şey yanar.

Debounce: 200 ms'lik sessizlik penceresi



Mekanik kontaklar kapanırken birkaç milisaniye titrer. Her titreşim ayrı bir falling edge, yani ayrı bir kesmedir: tek basışta state machine iki üç durum birden atlayabilir.

Somut: Kapı ziline düğmesi gevşek olsa bir basışta zil üç kez çalardı. Çözüm zili değiştirmek değil; "son çalıştan bu yana 200 ms geçmediyse duymazdan gel" demek.

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    static uint32_t last_tick = 0;
    uint32_t now = HAL_GetTick();

    if (GPIO_Pin != KEY_Pin)    { return; }
    if ((now - last_tick) < 200) { return; }

    last_tick = now;

    switch (led_state) { /* durum gecisi */
    }
}
```

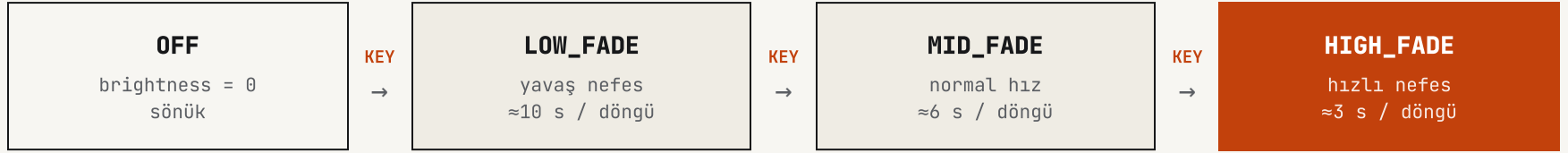
static burada zorunlu: fonksiyondan çıkınca değer silinmesin, bir sonraki kesmede hatırlansın. HAL_GetTick() ise SysTick sayacından açılıştan bu yana geçen milisaniyeyi döndürür.

06

State machine tasarımı

Dört durum, tek değişken, iki switch-case: biri geçişleri, diğeri davranışı yönetir.

Dört durum, döngüsel geçiş



----- KEY → başa dön (OFF): brightness = 0, fade_up = 1, fade_counter = 0 ile garanti sönük başlangıç -----

```
typedef enum {  
    LED_STATE_OFF = 0,  
    LED_STATE_LOW_FADE,  
    LED_STATE_MID_FADE,  
    LED_STATE_HIGH_FADE  
} LedState_t;
```

Neden enum? 0/1/2/3 yazmak yerine isim kullanmak kodu okunur yapar, derleyici yazım hatasını yakalar ve switch içinde eksik durum bırakırsanız uyarı verir.

Durumun PWM'e etkisi

DURUM	fade_step_ticks	1 KADEME SÜRESİ	TAM DÖNGÜ (0→100→0)
LED_STATE_OFF	—	—	brightness sabit 0
LED_STATE_LOW_FADE	500	$500 \times 100 \mu s = 50 \text{ ms}$	$\approx 10 \text{ s}$
LED_STATE_MID_FADE	300	$300 \times 100 \mu s = 30 \text{ ms}$	$\approx 6 \text{ s}$
LED_STATE_HIGH_FADE	150	$150 \times 100 \mu s = 15 \text{ ms}$	$\approx 3 \text{ s}$

Nasıl çalışıyor? Her TIM2 kesmesinde fade_counter bir artar. fade_step_ticks değerine ulaştınca brightness bir kademe değişir ve sayaç sıfırlanır.

Küçük değer (150) → brightness sık değişir → hızlı nefes.
Büyük değer (500) → seyrek değişir → yavaş nefes. Sayı büyüdükçe LED yavaşlar; sezgiye ters gelen kısım budur.

main.c özeti

```
/* Global degiskenler - hepsi volatile */
volatile LedState_t led_state = LED_STATE_OFF;
volatile uint8_t pwm_phase = 0; // 0-99
volatile uint8_t brightness = 0; // 0-100
volatile uint8_t fade_up = 1; // 1 = artiyor
volatile uint16_t fade_counter = 0;

/* main() icinde */
MX_GPIO_Init();
MX_TIM2_Init();
HAL_TIM_Base_Start_IT(&htim2);
while (1) { } // her sey kesmelerde

/* Buton kesmesi */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    /* debounce 200 ms + switch(led_state) */
}

/* TIM2 kesmesi, 10 kHz */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    /* 1) led_state'e gore fade_step_ticks sec
       2) fade_counter ile brightness'i kaydir
       3) pwm_phase < brightness ise yak, degilse sondur */
}
```

volatile NEDEN ŞART

Derleyici, bir değişkenin döngü içinde değişmediğini görürse onu register'a alıp bir daha bellekten okumaz. Ama bu değişkenleri **kesme** değiştiriyor; derleyicinin bundan haberi yok.

volatile "bu değişkeni her seferinde bellekten oku" demektir. Unutulursa LED bir durumda takıllı kalır; hata gözle görülür ama kodda görünmez.

while(1) neden boş? Tüm iş iki kesme fonksiyonunda. Ana döngü boş kalır; ileride güç tasarrufu için buraya __WFI() konabilir.

Test öncesi kontrol listesi

- ✓ HSI = dahili 16 MHz RC osilatör; PLL kapalıysa SYSCCLK = HSI.
- ✓ Kesme frekansı = $TIM2CLK \div (PSC+1) \div (ARR+1) = 16 \text{ MHz} \div 16 \div 100 = 10 \text{ kHz}$.
- ✓ APB1 prescaler 1 ise $TIMxCLK = PCLK1$; 1'den büyükse $TIMxCLK = 2 \times PCLK1$.
- ✓ PC13 aktif-low: pin LOW = LED yanık, pin HIGH = LED sönmük.
- ✓ Yazılımsal PWM: `pwm_phase < brightness` → pin LOW, yani LED yanık.
- ✓ Update Event, CNT = ARR anında donanım tarafından üretilir ve UIF'i set eder.
- ✓ EXTI bir GPIO pinini kesme kaynağına çevirir; NVIC kesmeyi CPU'ya iletir.
- ✓ Falling edge = HIGH→LOW; KEY basılınca PA0 GND'ye çekildiği için tetiklenir.
- ✓ Pull-up olmadan boştaki giriş pini floating kalır; PA0'a dahili pull-up şart.
- ✓ Debounce = mekanik sıçramayı zaman penceresiyle (200 ms) filtrelemek.
- ✓ volatile: ISR ile paylaşılan değişkende derleyici önbelleklemesini engeller.
- ✓ USER CODE BEGIN/END dışındaki her şey "Generate Code"ta yeniden yazılır.

08

Test: 12 soru

Cevaplar her sorunun hemen altında. Kendini gerçekten sınamak istiyorsan Tweaks panelinden "Cevapları göster"i kapat, sonra tekrar aç.

SORU 1

TIM2CLK = 16 MHz, PSC = 15, ARR = 99. TIM2 kesmesi hangi frekansta üretilir?

- A) 1 kHz
- B) 10 kHz
- C) 100 kHz
- D) 160 kHz

CEVAP · B

$16.000.000 \div 16 = 1 \text{ MHz}$; $1.000.000 \div 100 = 10 \text{ kHz}$. Her iki bölmede de "+1" unutulmamalı.

SORU 2

PC13 aktif-low bağlı. LED'i yakmak için hangi satır doğrudur?

- A) WritePin(..., GPIO_PIN_SET)
- B) WritePin(..., GPIO_PIN_RESET)
- C) TogglePin(...)
- D) ReadPin(...)

CEVAP · B

RESET pini 0 V'a çeker. LED'in anodu 3.3 V'ta olduğu için akım ancak o zaman geçer.

SORU 3

HCLK 16 MHz iken APB1 prescaler'ı 4 yapılsaydı TIM2CLK ne olurdu?

- A) 4 MHz
- B) 8 MHz
- C) 16 MHz
- D) 32 MHz

CEVAP · B

$PCLK1 = 16 \div 4 = 4$ MHz. Prescaler 1'den büyük olduğu için $TIMxCLK = 2 \times PCLK1 = 8$ MHz.

SORU 4

PA0'da dahili pull-up etkinleştirilmezse en olası sonuç nedir?

- A) Buton hiç çalışmaz
- B) Rastgele kesmeler oluşur
- C) LED sürekli yanar
- D) TIM2 durur

CEVAP · B

Boştaki pin floating kalır, çevresel gürültüyle rastgele HIGH→LOW geçişleri okur; durum kendi kendine değişir.

SORU 5

ARR'ı 99'dan 199'a çıkarırsam LED'de ne değişir? Sayılarla açıkla.

CEVAP

Kesme frekansı 10 kHz'den 5 kHz'e düşer ($1 \text{ MHz} \div 200$), callback her 200 μs 'de çalışır. pwm_phase hâlâ 0..99 döndüğü için PWM periyodu 20 ms, frekans 50 Hz olur; titreme sınırına yaklaşılr. Ayrıca tüm fade süreleri iki katına çıkar: HIGH_FADE $\approx 3 \text{ s}$ yerine $\approx 6 \text{ s}$.

SORU 6

Debounce eşiğini 200 ms yerine 20 ms yaparsam ne olur? Ya 2000 ms yaparsam?

CEVAP

20 ms: tipik kontak sıçraması 5-20 ms sürer, pencere dar kalır, bazı basışlar iki durum birden atlatır. 2000 ms: sıçrama kesin filtrelenir ama kullanıcı iki saniye içinde ikinci kez basamaz, arayüz takılmış gibi hissettirir. 200 ms ikisinin arasında güvenli bir orta noktadır.

SORU 7 – bu blok ne yapar?

```
fade_counter++;  
if (fade_counter ≥ fade_step_ticks) {  
    fade_counter = 0;  
    if (fade_up) { brightness++; }  
    else        { brightness--; }  
    if (brightness == 100) { fade_up = 0; }  
    if (brightness == 0)   { fade_up = 1; }  
}
```

CEVAP

Nefes alma efektinin motoru. Sayaç fade_step_ticks'e ulaştıkça brightness'ı bir artırır veya azaltır; 100'e varınca yön aşağı, 0'a inince yön yukarı döner. Yani brightness 0 ile 100 arasında üçgen dalga çizer.

SORU 8 – fark nedir?

```
HAL_TIM_Base_Start(&htim2);  
/* ...veya... */  
HAL_TIM_Base_Start_IT(&htim2);
```

Projede ikincisi kullanıldı. Birincisi kullanılsaydı LED ne yapardı?

CEVAP

Base_Start sayacı çalıştırır ama kesme üretmez; UIF set olur, NVIC'e gitmez. Callback hiç çağrılmaz, brightness ve pwm_phase hiç değişmez, LED sürekli sönük kalır. Sayaç arka planda döndüğü için hata "kod hiç çalışmıyor" gibi görünür.

Aşağıdaki iki kod parçasında birer hata var. Bul.

SORU 9

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    uint32_t last_tick = 0;
    uint32_t now = HAL_GetTick();

    if ((now - last_tick) < 200) { return; }
    last_tick = now;

    switch (led_state) { /* ... */ }
}
```

CEVAP

static eksik. last_tick her çağrıda 0'a döner, koşul asla sağlanmaz, debounce hiç çalışmaz. Ayrıca GPIO_Pin kontrolü de yok: başka bir EXTI hattı da bu callback'i tetikleyebilir.

SORU 10

```
uint8_t brightness = 0;
LedState_t led_state = LED_STATE_OFF;

/* ISR bunlari degistiriyor,
diger kod okuyor */
```

Derleyici optimizasyonu (-O2) açıkken ne olur?

CEVAP

volatile eksik. Derleyici değeri register'da tutup bellekten yeniden okumayabilir; ISR'ın yaptığı değişiklik görünmez olur. Hata çoğu zaman -O0'da ortaya çıkmaz, optimizasyon açılınca belirir.

SORU 11 – hesap zinciri

Bölen = PSC + ____

CK_CNT = 16 MHz ÷ ____ = 1 MHz

Kesme frekansı = 1 MHz ÷ ____ = 10 kHz

PWM frekansı = ____ Hz

CEVAP

1 · 16 · 100 · 100

SORU 12 – kavram eşleştirme

Pini kesme kaynağına çeviren birim: ____

Kesmeyi CPU'ya ileten denetleyici: ____

CNT = ARR anında üretilen olay: ____

ISR ile paylaşılan değişkenin niteleyicisi: ____

CEVAP

EXTI · NVIC · Update Event (UEV) · volatile

Kartta denemek için üç deney

PSC = 159 yap

CK_CNT 100 kHz'e, PWM 10 Hz'e düşer.
LED'in kırpmasını çıplak gözle gör;
PWM frekansının neden yüksek
tutulduğunu bir kerede anlarsın.

volatile'ları kaldır

-02 ile derle. Efektin bozulup
bozulmadığını izle, sonra assembly
çıktısında değişkenin bellekten mi
register'dan mı okunduğuna bak.

Debounce'ı kapat

200 ms kontrolünü kaldır ve butona
bas. Tek basışta iki üç durum birden
atlandığını gör; sıçrama artık soyut
bir kavram olmaktan çıkar.